```
      MarketServiceHandler.withServerReflection(
        new MarketServiceImplSharding())                          ❶

    val port =
      system.settings.config.getInt("services.market.port")
    val host = system.settings.config.getString("services.host")

    Http().newServerAt(host, port).bind(marketService)            ❷
  }

}
```

❶ Selects MarketServiceImplSharding

❷ Starts the server and binds the service

All the inputs in the `init` method of this server are implicit. And thanks to that, the sharding is also passed implicitly to the service implementation constructor:

```
class MarketServiceImplSharding(implicit sharding: ClusterSharding)
```

NOTE Using `implicit` allows you to avoid passing the same input to chained functions repeatedly. This is less work and less noise—although you could argue that this is less explicit and therefore more complex.

The market server is added to the entry point of the application, and with it, the market service.

Listing 12.16 Entry point of the betting-house application

```
object Main {

  val log = LoggerFactory.getLogger(Main + "")

  def main(args: Array[String]): Unit = {
    implicit val system =
      ActorSystem[Nothing](Behaviors.empty, "betting-house")
    try {
      implicit val sharding = ClusterSharding(system)             ❶
```